

Rainfall Data Analysis Assignment (100 Marks, Due 6th July 2025)

You are provided with a rainfall dataset recording monthly measurements from various weather stations across different regions. The dataset includes rainfall (mm), number of rainy days, maximum and minimum temperatures, and whether the station is in an urban or rural area.

Dataset columns:

- `Region` : Name of the region (e.g., North, South, East, West)
- `Year` : Year of measurement
- `Month` : Month (e.g., Jan, Feb, ..., Dec)
- `Station` : Weather station name/ID (e.g., StationA, StationB)
- `Rainfall_mm` : Rainfall amount in millimeters
- `RainyDays` : Number of days in the month with rain
- `MaxTemp_C` : Maximum temperature recorded in that month (°C)
- `MinTemp_C` : Minimum temperature recorded in that month (°C)
- `UrbanRural` : "Urban" or "Rural"

Part 1: Data Exploration (15 Marks)

1. Load the data into a pandas DataFrame. (2 Marks)
2. Display the first 5 rows of the DataFrame. (2 Marks)
3. Check for and handle any missing values. (6 Marks)
4. Provide summary statistics for all numeric columns. (5 Marks)

```
In [49]: #Write your code here
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("Rainfall.csv")
df.head()

cols = list(df.columns)
new_cols = list(cols[0].split(','))
new_cols

rain = df[cols[0]].str.split(',', expand=True)
rain.columns = new_cols

print(rain.isnull().sum())

numeric_cols = ['Rainfall_mm', 'RainyDays', 'MaxTemp_C', 'MinTemp_C']
rain[numeric_cols] = rain[numeric_cols].apply(pd.to_numeric, errors='coerce')

month_order = ['Jan', 'Feb', 'Mar']
```

```
rain['Month'] = pd.Categorical(rain['Month'], categories=month_order, ordered=True)

rain.to_csv('Rainfall_new.csv')

print(rain)
rain.describe()
```

```
Region      0
Year        0
Month       0
Station     0
Rainfall_mm 0
RainyDays   0
MaxTemp_C   0
MinTemp_C   0
UrbanRural  0
```

```
dtype: int64
```

	Region	Year	Month	Station	Rainfall_mm	RainyDays	MaxTemp_C	MinTemp_C	\
0	North	2023	Jan	StationA	56.2	9	12.1	2.5	
1	North	2023	Feb	StationA	72.1	12	14.7	3.0	
2	North	2023	Jan	StationB	60.5	10	11.2	1.8	
3	South	2023	Jan	StationC	81.0	13	19.6	7.2	
4	South	2023	Feb	StationC	63.4	11	20.1	7.7	
5	East	2023	Jan	StationD	45.7	8	15.4	5.6	
6	East	2023	Feb	StationD	59.6	10	17.9	6.2	
7	West	2023	Jan	StationE	77.4	12	16.8	4.9	
8	West	2023	Feb	StationE	84.2	13	18.2	5.3	
9	North	2023	Mar	StationB	61.5	11	15.0	3.2	

```
UrbanRural
0      Urban
1      Urban
2      Rural
3      Urban
4      Urban
5      Rural
6      Rural
7      Urban
8      Urban
9      Rural
```

Out[49]:

	Rainfall_mm	RainyDays	MaxTemp_C	MinTemp_C
count	10.000000	10.000000	10.000000	10.000000
mean	66.160000	10.900000	16.100000	4.740000
std	12.154121	1.663333	2.982542	2.029888
min	45.700000	8.000000	11.200000	1.800000
25%	59.825000	10.000000	14.775000	3.050000
50%	62.450000	11.000000	16.100000	5.100000
75%	76.075000	12.000000	18.125000	6.050000
max	84.200000	13.000000	20.100000	7.700000

Part 2: Grouping and Aggregation (30 Marks)

1. For each **Station**, compute: (6 Marks)

- Total rainfall
- Average number of rainy days
- Average maximum temperature
- Average minimum temperature

In [50]: *#Write your code here*

```
rain_station = rain.groupby('Station')

rain_station.agg({'Rainfall_mm': ['sum'],
                  'RainyDays': ['mean'],
                  'MaxTemp_C': ['mean'],
                  'MinTemp_C': ['mean']})
```

Out[50]:

	Rainfall_mm	RainyDays	MaxTemp_C	MinTemp_C
	sum	mean	mean	mean
Station				
StationA	128.3	10.5	13.40	2.75
StationB	122.0	10.5	13.10	2.50
StationC	144.4	12.0	19.85	7.45
StationD	105.3	9.0	16.65	5.90
StationE	161.6	12.5	17.50	5.10

2. For each **Region & UrbanRural** combination, calculate: (6 Marks)

- Average rainfall
- Average maximum temperature
- Average number of rainy days

In [51]: *#Write your code here*

```
reg_urbanRural = rain.groupby(['Region', 'UrbanRural'])
reg_urbanRural.agg({'Rainfall_mm' : 'mean',
                    'MaxTemp_C' : 'mean',
                    'RainyDays' : 'mean'})
```

Out[51]:

		Rainfall_mm	MaxTemp_C	RainyDays
Region	UrbanRural			
East	Rural	52.65	16.65	9.0
North	Rural	61.00	13.10	10.5
	Urban	64.15	13.40	10.5
South	Urban	72.20	19.85	12.0
West	Urban	80.80	17.50	12.5

3. For each **Month** and **Region**, calculate: (6 Marks)

- Total rainfall
- Mean rainfall
- Minimum and maximum rainfall

In [52]: *#Write your code here*

```
month_region = rain.groupby(['Month','Region'])
month_region.agg({'Rainfall_mm' : ['sum', 'mean', 'min', 'max']})
```

C:\Users\User\AppData\Local\Temp\ipykernel_21788\3213127648.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
month_region = rain.groupby(['Month','Region'])
```

Out[52]:

		Rainfall_mm			
		sum	mean	min	max
Month	Region				
Jan	East	45.7	45.70	45.7	45.7
	North	116.7	58.35	56.2	60.5
	South	81.0	81.00	81.0	81.0
	West	77.4	77.40	77.4	77.4
Feb	East	59.6	59.60	59.6	59.6
	North	72.1	72.10	72.1	72.1
	South	63.4	63.40	63.4	63.4
	West	84.2	84.20	84.2	84.2
Mar	East	0.0	NaN	NaN	NaN
	North	61.5	61.50	61.5	61.5
	South	0.0	NaN	NaN	NaN
	West	0.0	NaN	NaN	NaN

4. Find the **Region and Month** with the highest recorded rainfall in the dataset. (4 Marks)

In [53]: *#Write your code here*

```
highest = month_region['Rainfall_mm'].max().idxmax()

month, region = highest

print(f"Month with the highest rainfall is in {region} on {month}")
```

Month with the highest rainfall is in West on Feb

5. For each **Year** and **UrbanRural** group, calculate: (4 Marks)

- Average rainfall
- Minimum rainfall
- Maximum rainfall

```
In [54]: #Write your code here
year_urbanRural = rain.groupby(['Year', 'UrbanRural'])
year_urbanRural.agg({'Rainfall_mm' : ['mean', 'min', 'max']}).round(2)
```

```
Out[54]:
```

		Rainfall_mm		
		mean	min	max
Year	UrbanRural			
2023	Rural	56.82	45.7	61.5
	Urban	72.38	56.2	84.2

6. For each **Region**, calculate the average temperature range per month (i.e., average of MaxTemp_C - MinTemp_C). (4 Marks)

```
In [55]: #Write your code here
rain['TempRange'] = rain['MaxTemp_C'] - rain['MinTemp_C']
month_region = rain.groupby(['Region', 'Month'])
month_region['TempRange'].mean()
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_21788\2821905047.py:3: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
  month_region = rain.groupby(['Region', 'Month'])
```

```
Out[55]:
```

Region	Month	
East	Jan	9.8
	Feb	11.7
	Mar	NaN
North	Jan	9.5
	Feb	11.7
	Mar	11.8
South	Jan	12.4
	Feb	12.4
	Mar	NaN
West	Jan	11.9
	Feb	12.9
	Mar	NaN

Name: TempRange, dtype: float64

Part 3: Data Visualization (matplotlib) (40 Marks)

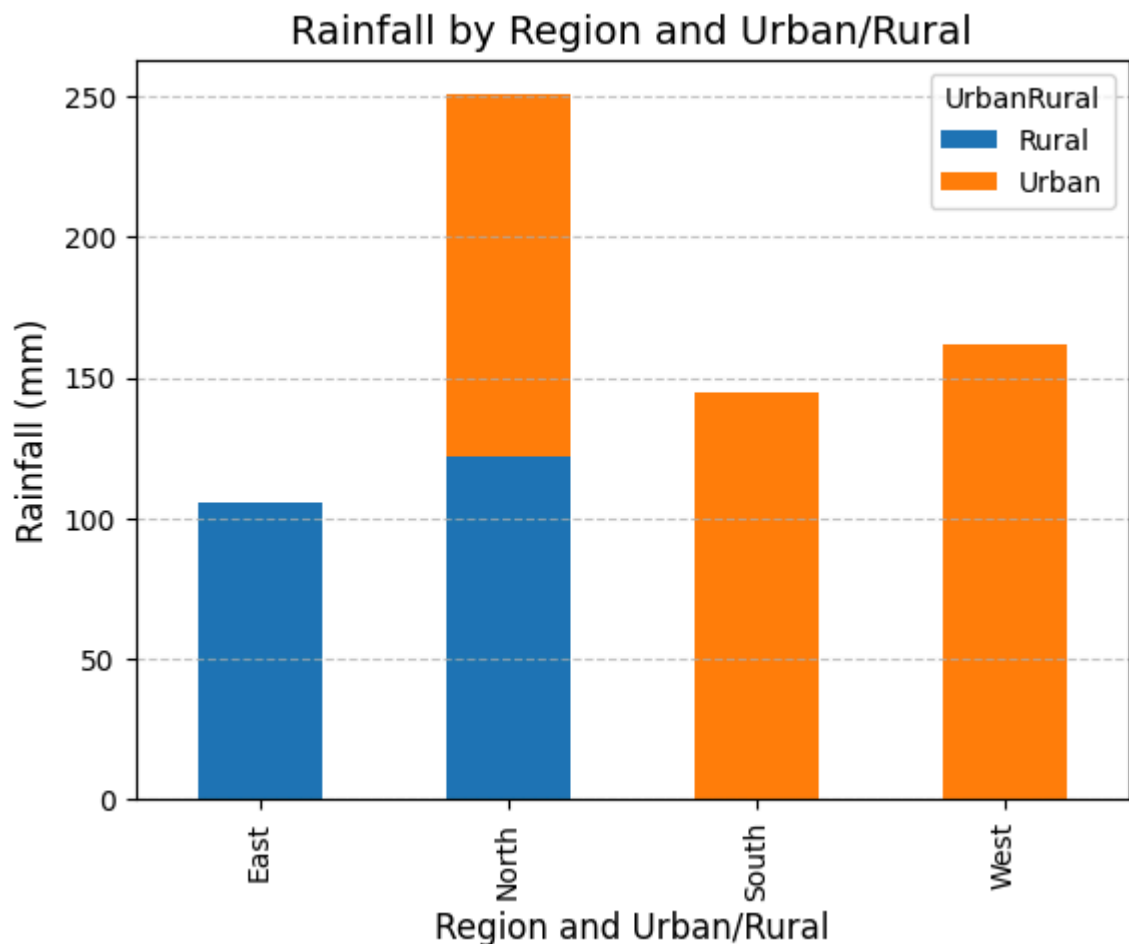
1. **Bar Chart:** (8 Marks)

- Plot total rainfall by region, colored by Urban/Rural.
- Add value labels, gridlines, axis labels, title, and adjust bar width for clarity.

```
In [56]: #Write your code here
total_rain = reg_urbanRural['Rainfall_mm'].sum().unstack()
print(total_rain)

# Create a bar chart
fig, ax = plt.subplots()
total_rain.plot(kind='bar', ax=ax, stacked = True)
plt.title('Rainfall by Region and Urban/Rural', fontsize = 14)
plt.xlabel('Region and Urban/Rural', fontsize = 12)
plt.ylabel('Rainfall (mm)', fontsize = 12)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.savefig('bar.png')
plt.show()
```

UrbanRural	Rural	Urban
Region		
East	105.3	NaN
North	122.0	128.3
South	NaN	144.4
West	NaN	161.6



2. Grouped Bar Chart: (6 Marks)

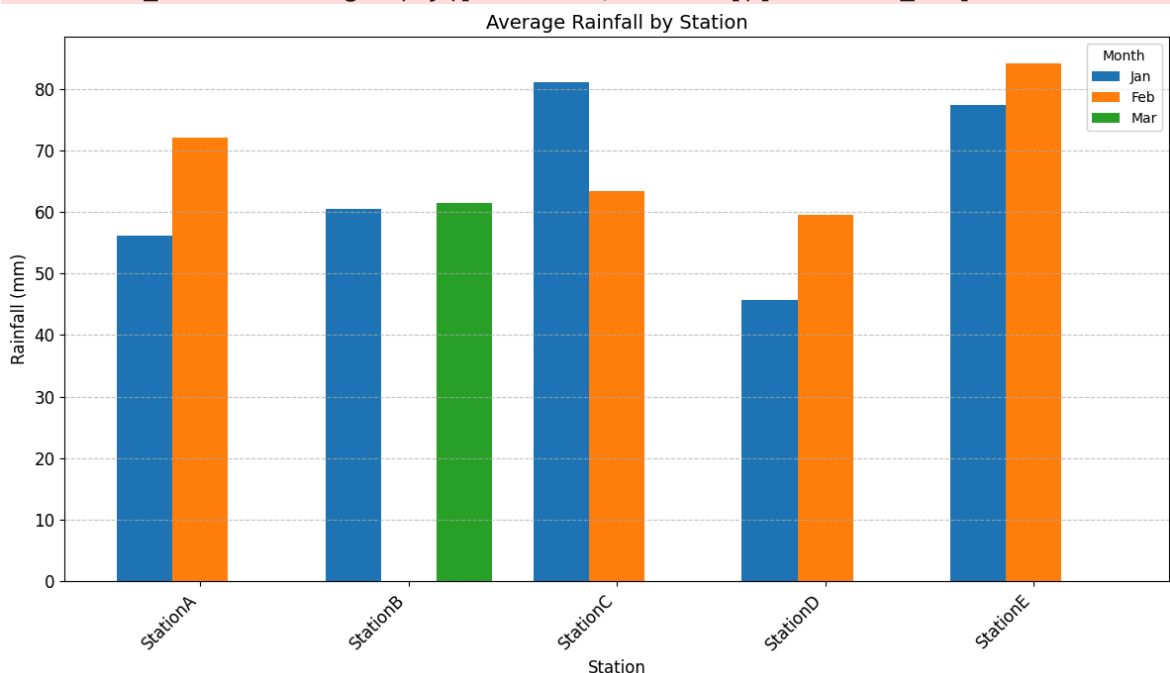
- Show average monthly rainfall for each station.
- Use different colors for each station, include a legend, rotate x-tick labels, and increase figure size for readability.

```
In [57]: #Write your code here
station_month = rain.groupby(['Station', 'Month'])['Rainfall_mm']
```

```
fig, ax = plt.subplots(figsize=(12, 7))
station_month.mean().unstack().plot(kind='bar', ax=ax, width=0.8)
plt.title('Average Rainfall by Station', fontsize = 14)
plt.xlabel('Station', fontsize = 12)
plt.ylabel('Rainfall (mm)', fontsize = 12)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.xticks(rotation=45, ha='right', fontsize = 12)
plt.yticks(fontsize = 12)
plt.tight_layout()
plt.savefig('grpuedbar.png')
plt.show()
```

C:\Users\User\AppData\Local\Temp\ipykernel_21788\403597836.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
station_month = rain.groupby(['Station', 'Month'])['Rainfall_mm']
```



3. Line Plot: (6 Marks)

- Show the trend of rainfall over months for at least two selected stations.
- Use markers, dashed lines, highlight the month with maximum rainfall, and format the x-axis for months.

In [58]: *#Write your code here*

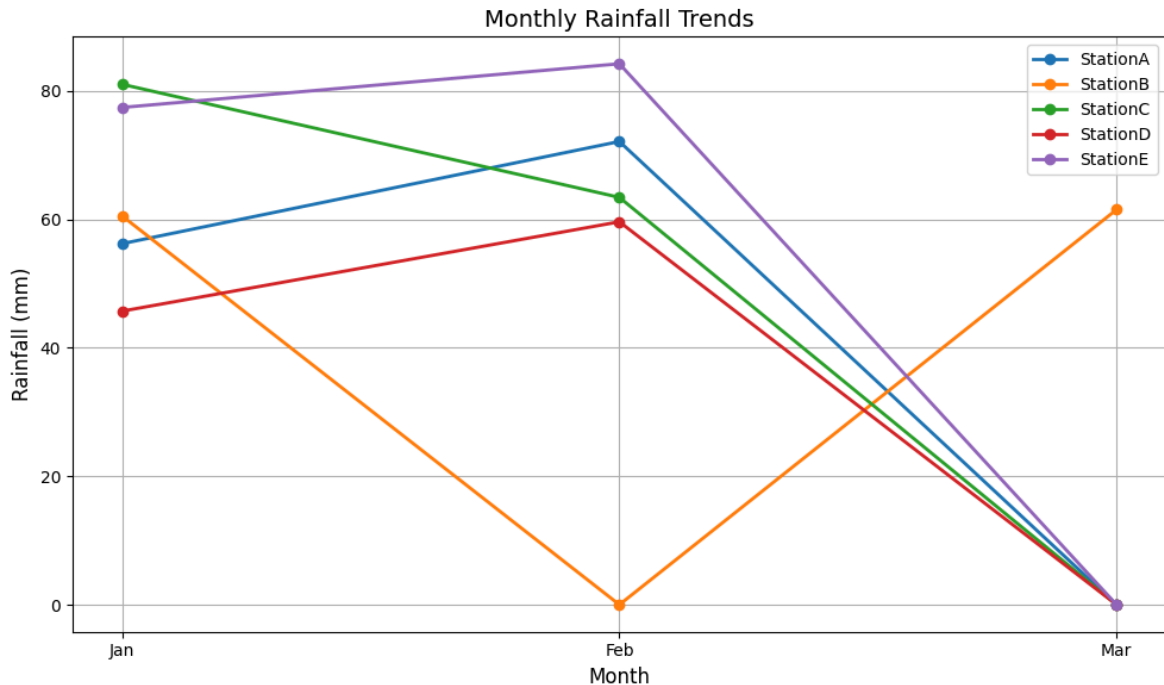
```
rainfall_trend = station_month.mean().unstack()
rainfall_trend.fillna(0, inplace=True)

station = rainfall_trend.index

# Simple plot
plt.figure(figsize=(10, 6))

# Plot each station
for station in station:
    plt.plot(rainfall_trend.columns, rainfall_trend.loc[station],
             marker='o', label=station, linewidth=2)
```

```
plt.xlabel('Month' , fontsize = 12)
plt.ylabel('Rainfall (mm)',fontsize = 12)
plt.title('Monthly Rainfall Trends', fontsize = 14)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig('line.png')
plt.show()
```



4. Boxplot: (6 Marks)

- Display rainfall distributions by region and by Urban/Rural.
- Use custom box colors, highlight outliers, and set y-axis limits.

```
In [59]: #Write your code here
# Get unique regions and urban/rural categories
regions = rain['Region'].unique()
urb = rain['UrbanRural'].unique()

# Create two plots side by side
fig, ax = plt.subplots(1, 2, figsize=(14, 10))

# First plot: Rainfall by Region
box1 = ax[0].boxplot([rain[rain['Region'] == region]['Rainfall_mm'] for region in regions],
                    patch_artist=True) # This is needed for colors
ax[0].set_xticklabels(regions)
ax[0].set_title('Rainfall by Region', fontsize = 14)
ax[0].set_ylabel('Rainfall (mm)', fontsize = 12)
ax[0].set_xlabel('Region', fontsize = 12)
ax[0].set_ylim(0, rain['Rainfall_mm'].max()+10)

# Add custom colors to boxes
colors = ['lightblue', 'lightgreen', 'lightcoral', 'lightyellow']
for box, color in zip(box1['boxes'], colors):
    box.set_facecolor(color)

# Make outliers red and bigger
for outlier in box1['fliers']:
    outlier.set_color('red')
    outlier.set_size(150)
```



```

outlier.set_color('red')
outlier.set_markersize(8)

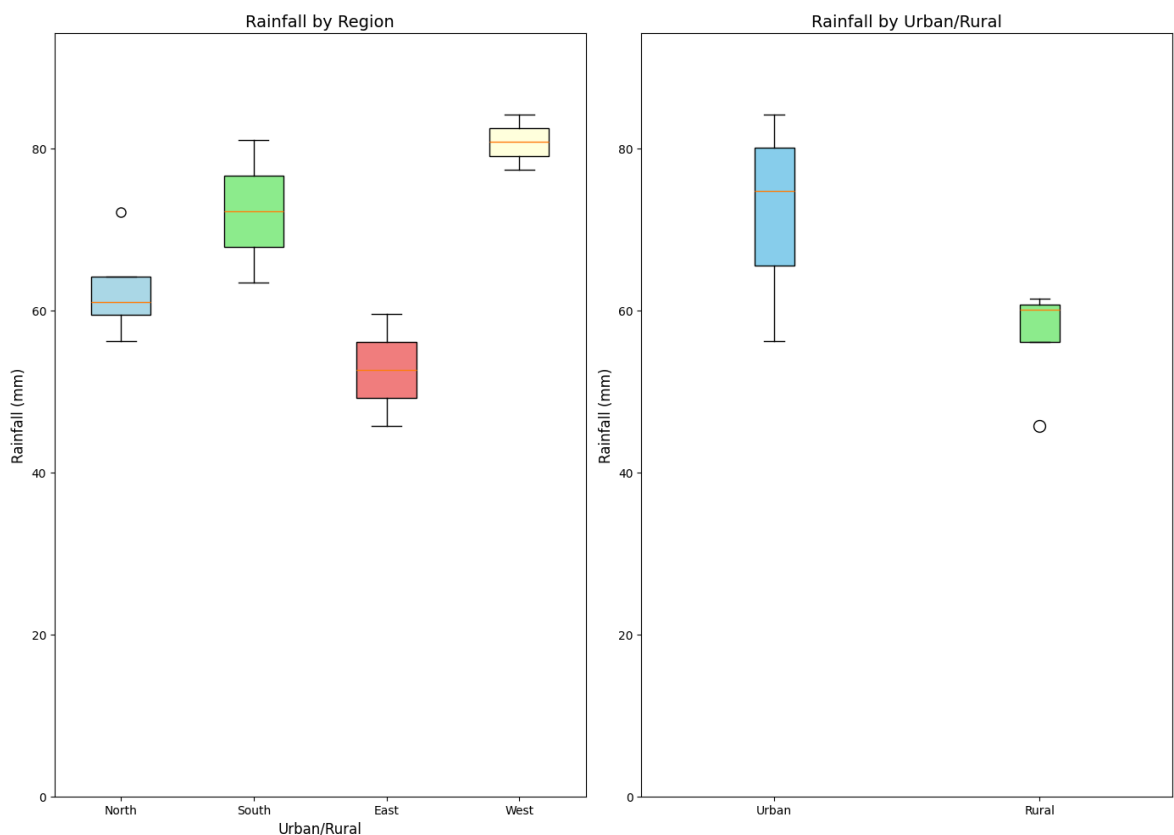
# Second plot: Rainfall by Urban/Rural
box2 = ax[1].boxplot([rain[rain['UrbanRural'] == u]["Rainfall_mm"] for u in urb]
                    patch_artist=True) # This is needed for colors
ax[1].set_xticklabels(urb)
ax[1].set_title('Rainfall by Urban/Rural', fontsize = 14)
ax[1].set_ylabel('Rainfall (mm)', fontsize = 12)
ax[0].set_xlabel('Urban/Rural', fontsize = 12)
ax[1].set_ylim(0, rain['Rainfall_mm'].max()+10)

# Add custom colors to boxes
colors2 = ['skyblue', 'lightgreen']
for box, color in zip(box2['boxes'], colors2):
    box.set_facecolor(color)

# Make outliers red and bigger
for outlier in box2['fliers']:
    outlier.set_color('blue')
    outlier.set_markersize(10)

plt.tight_layout()
plt.savefig('box.png')
plt.show()

```

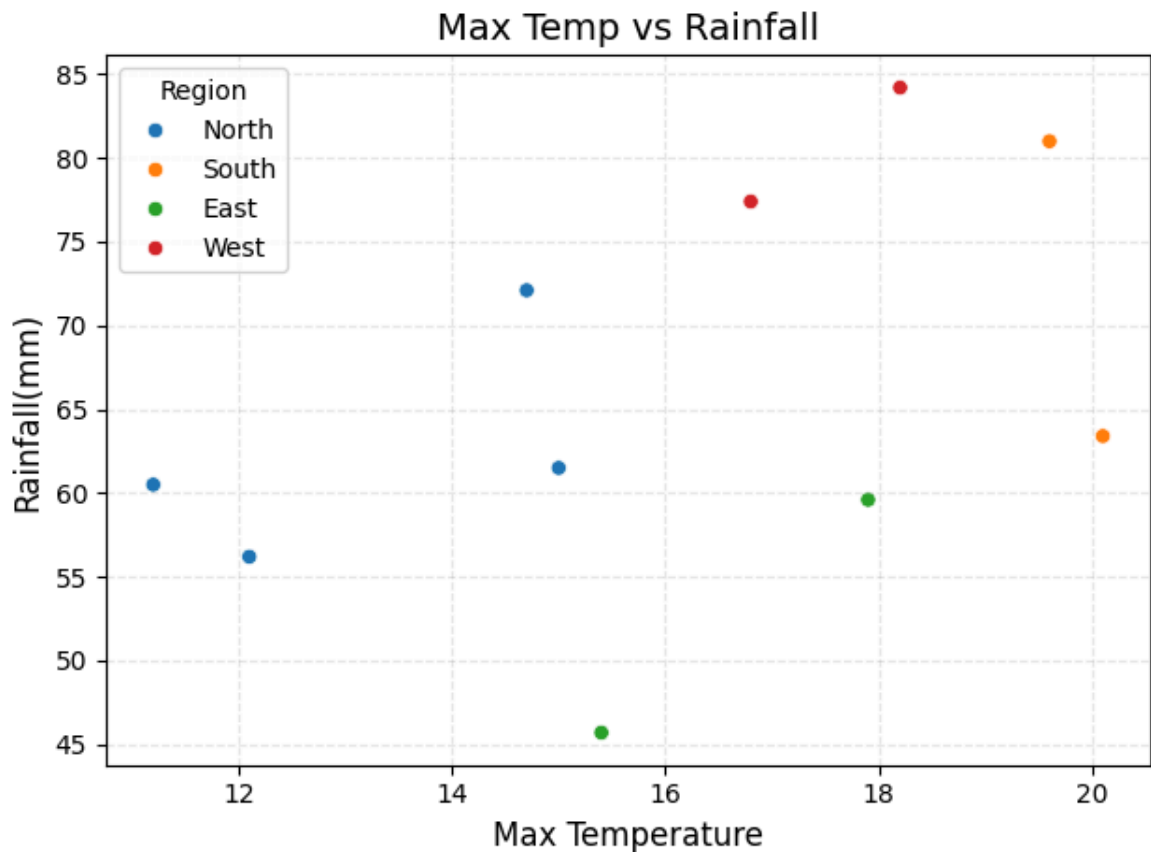


5. Scatter Plot: (6 Marks)

- Visualize the relationship between rainfall and maximum temperature, colored by region.
- Add axis labels, legend, gridlines, and an appropriate title.

In [60]: *#Write your code here*

```
sns.scatterplot(data = rain, x = 'MaxTemp_C', y = 'Rainfall_mm', hue = 'Region')
plt.title("Max Temp vs Rainfall", fontsize = 14)
plt.xlabel("Max Temperature", fontsize = 12)
plt.ylabel("Rainfall(mm)", fontsize = 12)
plt.grid(linestyle = '--', alpha = 0.3)
plt.tight_layout()
plt.savefig('scatter.png')
plt.show()
```



6. Heatmap:

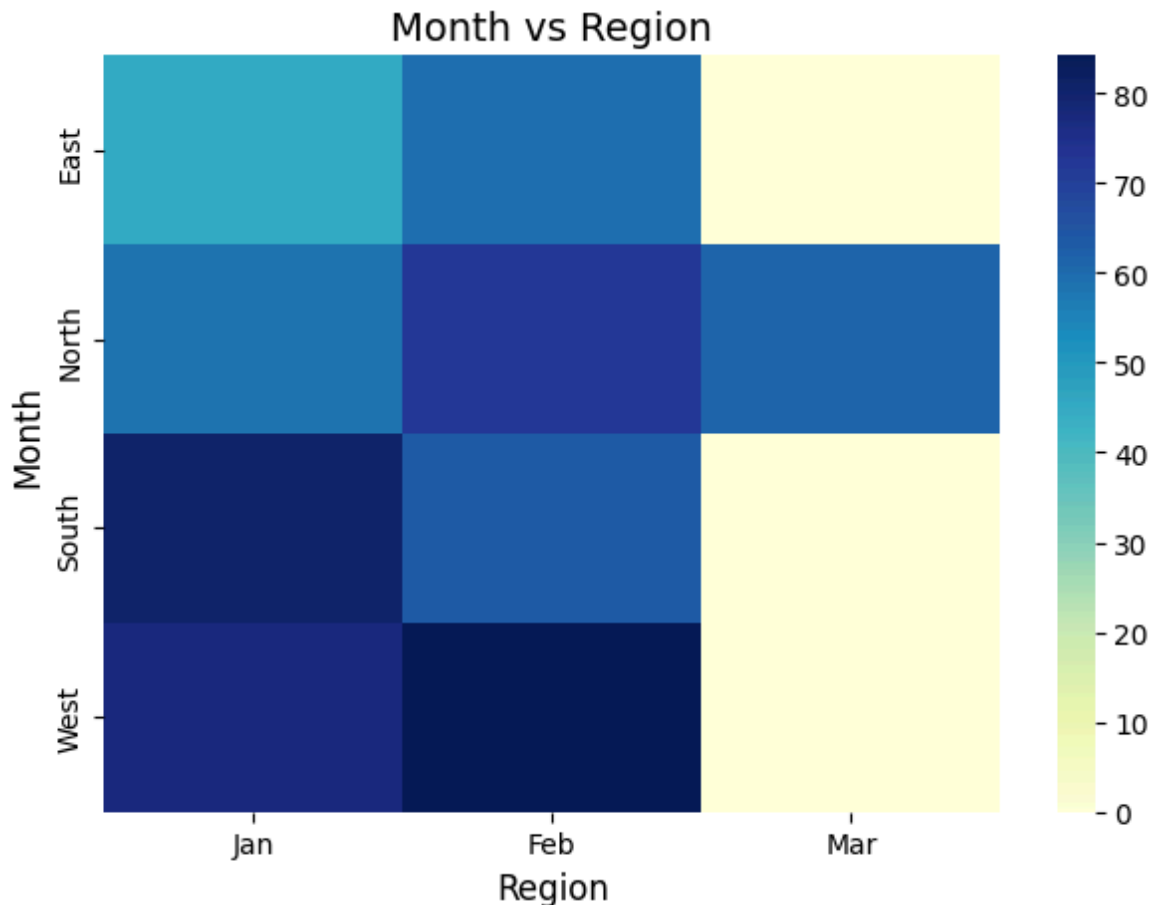
- Create a heatmap showing mean monthly rainfall for each region (Month on x-axis, Region on y-axis).
- Add color bar, axis labels, and title.

In [61]: *#Write your code here*

```
avg_month_region = month_region['Rainfall_mm'].mean()
avg_month_region.fillna(0, inplace=True)
print(avg_month_region)
sns.heatmap(avg_month_region.unstack(), cmap="YlGnBu")
plt.title("Month vs Region", fontsize = 14)
plt.xlabel("Region", fontsize = 12)
plt.ylabel("Month", fontsize = 12)
plt.tight_layout()
plt.savefig('heatmap.png')
plt.show()
```

Region	Month	
East	Jan	45.70
	Feb	59.60
	Mar	0.00
North	Jan	58.35
	Feb	72.10
	Mar	61.50
South	Jan	81.00
	Feb	63.40
	Mar	0.00
West	Jan	77.40
	Feb	84.20
	Mar	0.00

Name: Rainfall_mm, dtype: float64



7. Formatting (for all plots): (8 Marks)

- Use descriptive titles, axis labels, and legends.
- Increase font size for readability.
- Save each figure as an image file (.png).
- Use `plt.tight_layout()` for proper spacing.

In [62]: *#Write your code here*

All the code are written at all the plots before according to this question's

Part 4: Bonus Tasks (15 Marks)

1. Annotate at least one plot with the value and label of the highest/lowest point. (5 Marks)

```
In [63]: #Write your code here

# Calculate the average monthly rainfall for each station
average_rainfall_by_station_month = rain.groupby(['Station', 'Month'])['Rainfall']

# Create the figure and axes for the plot
fig, ax = plt.subplots(figsize=(14, 8))

# Plotting the grouped bar chart
average_rainfall_by_station_month.plot(kind='bar', ax=ax, width=0.8)

# Add a title to the plot
plt.title('Average Monthly Rainfall for Each Station', fontsize=16)

# Add Labels for the x and y axes
plt.xlabel('Station', fontsize=14)
plt.ylabel('Average Rainfall (mm)', fontsize=14)

# Rotate x-tick labels for readability
plt.xticks(rotation=45, ha='right', fontsize=12)

# Add a Legend
plt.legend(title='Month', bbox_to_anchor=(1.05, 1), loc='upper left', borderaxes

# Add a grid for better readability of values
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Adjust layout to prevent labels from being cut off
plt.tight_layout(rect=[0, 0, 0.85, 1])

# --- Annotating Highest and Lowest Points using ax.text() ---

# Find the overall max and min rainfall values and their corresponding indices
max_val = average_rainfall_by_station_month.max().max()
min_val = average_rainfall_by_station_month.min().min()

# Get the Location (Station and Month) for the highest value
max_month_series = average_rainfall_by_station_month.max()
max_month = max_month_series.idxmax()
max_station_series = average_rainfall_by_station_month[max_month]
max_station = max_station_series.idxmax()

# Get the Location (Station and Month) for the lowest value
min_month_series = average_rainfall_by_station_month.min()
min_month = min_month_series.idxmin()
min_station_series = average_rainfall_by_station_month[min_month]
min_station = min_station_series.idxmin()

# Get the index of the highest/lowest station on the x-axis
x_pos_max_station = list(average_rainfall_by_station_month.index).index(max_stat
x_pos_min_station = list(average_rainfall_by_station_month.index).index(min_stat

# Iterate over the bar containers to find the specific bar for annotation
for container in ax.containers:
    month_name = container.get_label()
    if month_name == max_month:
```

```

for bar in container:
    if bar.get_x() + bar.get_width()/2 > x_pos_max_station - 0.5 and bar
    if bar.get_height() == max_val:
        max_bar_x = bar.get_x() + bar.get_width() / 2
        max_bar_y = bar.get_height()
        ax.text(max_bar_x, max_bar_y + 5, # Slightly above the bar
                f'{max_val:.1f}',
                ha='center', va='bottom',
                fontsize=9, color='red', fontweight='bold',
                bbox=dict(boxstyle='round,pad=0.2', fc='yellow', ec=

elif month_name == min_month:
    for bar in container:
        if bar.get_x() + bar.get_width()/2 > x_pos_min_station - 0.5 and bar
        if bar.get_height() == min_val:
            min_bar_x = bar.get_x() + bar.get_width() / 2
            min_bar_y = bar.get_height()
            ax.text(min_bar_x, min_bar_y - 5, # Slightly below the bar
                    f'{min_val:.1f}',
                    ha='center', va='top',
                    fontsize=9, color='blue', fontweight='bold',
                    bbox=dict(boxstyle='round,pad=0.2', fc='lightblue',

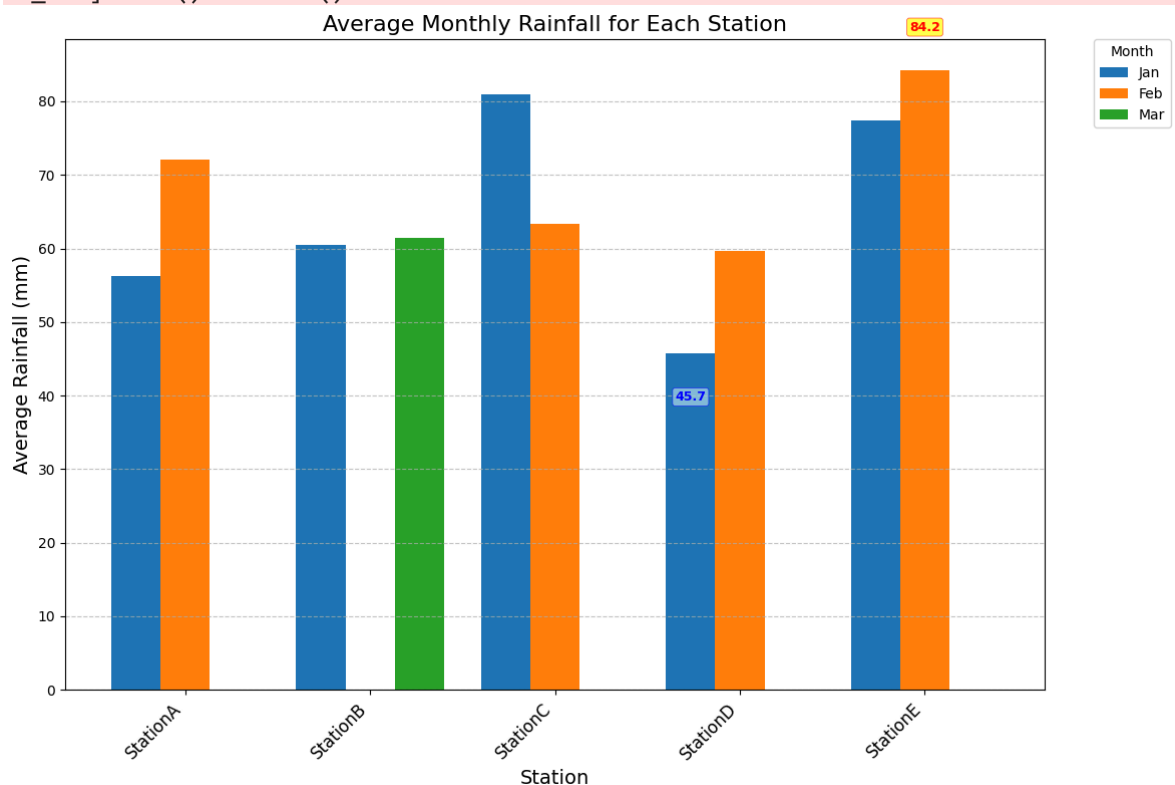
# Save the figure
plt.savefig('grouped_bar_chart_rainfall_text_annotated.png')

# Display the plot
plt.show()

```

C:\Users\User\AppData\Local\Temp\ipykernel_21788\3578212126.py:4: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
average_rainfall_by_station_month = rain.groupby(['Station', 'Month'])['Rainfall_mm'].mean().unstack()
```



2. Create a **facet/grid plot** comparing rainfall trends by region and Urban/Rural status.
(5 Marks)

```
In [68]: #Write your code here
import matplotlib.dates as mdates

# Create a 'Date' column by combining 'Year' and 'Month'
month_mapping = {
    'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun': 6,
    'Jul': 7, 'Aug': 8, 'Sep': 9, 'Oct': 10, 'Nov': 11, 'Dec': 12
}
rain['Month_Num'] = rain['Month'].map(month_mapping)
rain['Date'] = pd.to_datetime(rain['Year'].astype(str) + '-' + rain['Month_Num'])

# Group by Date, Region, and UrbanRural and calculate the mean rainfall
df_grouped = rain.groupby(['Date', 'Region', 'UrbanRural'])['Rainfall_mm'].mean()

# Create the facet/grid plot using seaborn.relplot
# We use relplot with kind='line' to show trends over time
# Create the facet/grid plot using seaborn.relplot
g = sns.relplot(
    data=df_grouped,
    x='Date',
    y='Rainfall_mm',
    col='UrbanRural',
    row='Region',
    kind='line',
    height=3, aspect=1.5,
    hue='UrbanRural',
    facet_kws={'sharey': False}
)

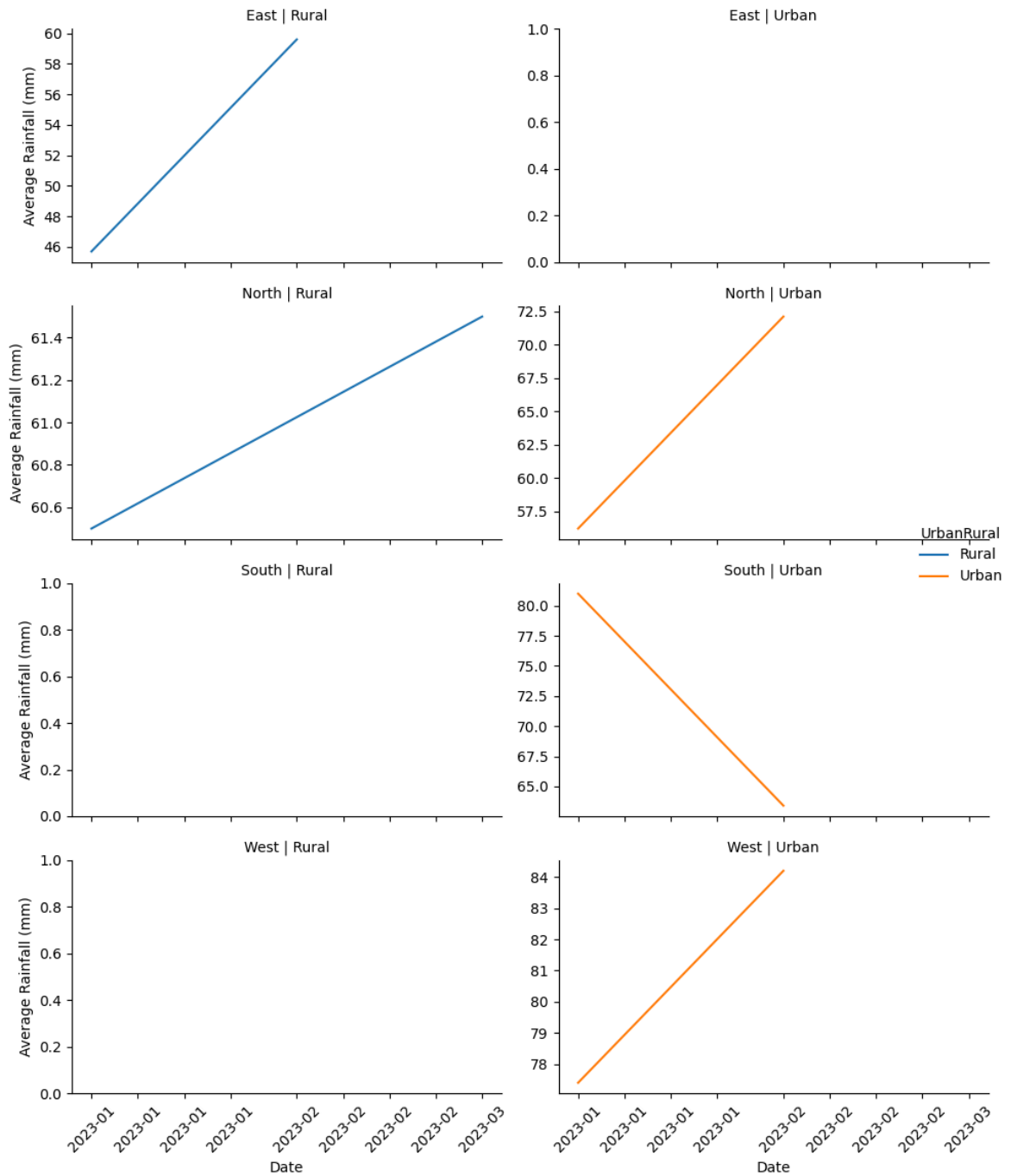
# Set titles and labels for clarity
g.set_axis_labels("Date", "Average Rainfall (mm)")
g.set_titles(col_template="{col_name}", row_template="{row_name}")
plt.suptitle('Average Rainfall Trends by Region and Urban/Rural Status', y=1.02)

# Format x-axis date labels
# Iterate through each subplot to apply date formatting
for ax in g.axes.flat:
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m')) # Format as YYYY
    ax.tick_params(axis='x', rotation=45) # Rotate for better readability

plt.tight_layout()
plt.show()

# Save the plot
plt.savefig('rainfall_trends_sns_relplot_labeled_date.png')
plt.close()
```

Average Rainfall Trends by Region and Urban/Rural Status



3. Write a brief summary (in a Markdown cell) of your main insights from the data and visualizations. (5 Marks)

Write your summary here